CSSR for

Culturally Sensitive Social Robotics for Africa

D4.3.2 Speech Event

Due date: **31/03/2024**
Submission Date: **20/02/2025**
Revision Date: **07/05/2026**

Start date of project: **01/07/2023** Duration: **36 months**

Lead organisation for this deliverable: **Carnegie Mellon University Africa**

Responsible Person: **Clifford Onyonka** Revision: **1.4**

Project funded by the African Engineering and Technology Network (Afretec) Inclusive Digital Transformation Research Grant Programme		
Dissemination Level		
PU	Public	**PU**
PP	Restricted to other programme participants (including Afretec Administration)	
RE	Restricted to a group specified by the consortium (including Afretec Administration)	
CO	Confidential, only for members of the consortium (including Afretec Administration)	

Executive Summary

Deliverable D4.3.2 concerns the results of Task 4.3.2, a task whose objective was to train, test, and finally deploy a speech-to-text model on a ROS node that will enable speech utterances in Kinyarwanda and English languages captured by Pepper's microphones to be transcribed into written text.

This report details the output of each phase of the software development process used in the fulfillment of Deliverable D4.3.2. The requirements definition section specifies the functional requirements of the users of Speech Event, the module specification section outlines the functional characteristics of Speech Event, the interface design section outlines the specification of the inputs and outputs of Speech Event, the module design section describes the deep neural networks that perform speech recognition of Kinyarwanda and English utterances, the testing section shows the results and descriptions of unit and end-to-end tests of Speech Event, and the user manual section outlines how to build and run Speech Event.

Contents

1 Introduction

Conversational systems are developed using the following main components: automatic speech recognition (speech-to-text), natural language understanding, dialogue manager, natural language generation, and text-to-speech. Speech-to-text converts an audio signal containing spoken speech utterances to written text transcriptions, natural language understanding analyses the transcribed text to extract meaning, the dialogue manager generates a response based on the inferred meaning of the text, natural language generation formulates text based on the response generated by the dialogue manager, and text-to-speech synthesises spoken speech utterances to be played to the other agent(s) in the conversation cycle [1]. Such a system is displayed in Fig 1.

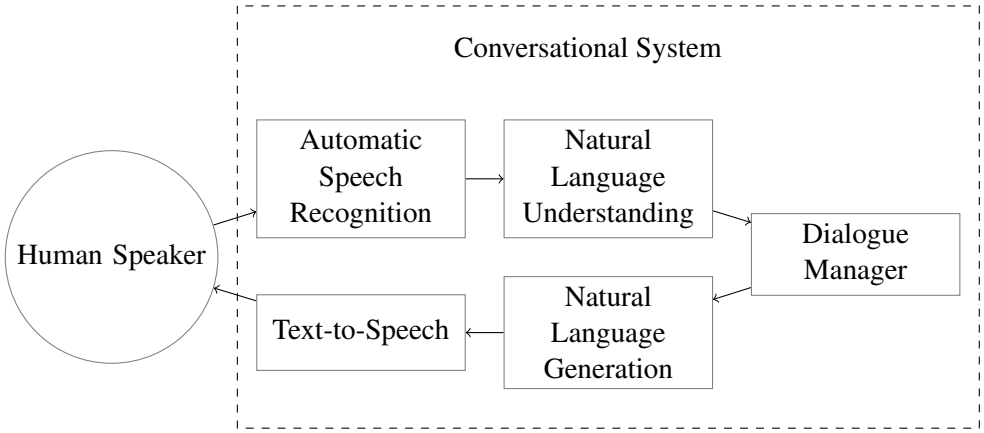

Figure 1: A diagrammatic representation of a conversational system [1]

The Speech Event module only handles one component of the conversational system described in the previous paragraph, that is, speech-to-text (which is referred to as automatic speech recognition in Fig 1). It is worth noting that the Culturally Sensitive Social Robotics for Africa (CSSR4Africa) project does not implement a full fledged conversational system, and therefore the following components are not implemented in the CSSR4Africa project: natural language understanding, dialogue management, and natural language generation. Instead, the Behaviour Controller, which directs the robot mission within a certain scenario such as a lab tour, acts as a proxy that approximates the functionality of all these three unimplemented components. The Behaviour Controller also determines the language that the Speech Event module will transcribe, reading it from the Culture Knowledge Base and then passing it to Speech Event via the `/speechEvent/set_language` ROS service that is advertised by Speech Event. The languages that Speech Event supports are Kinyarwanda and English.

2 Requirements Definition

A running Speech Event ROS node performs one main function - Kinyarwanda and English speech recognition. An audio signal is received via the `/soundDetection/signal` ROS topic, and Speech Event transcribes any speech utterances detected in the signal to either Kinyarwanda or English. This transcription process is initiated via the `/speechEvent/recognise_speech_action` ROS action.

In order to successfully perform this function, the following functional requirements are fulfilled by Speech Event:

1. Await a transcription request on the `/speechEvent/recognise_speech_action` ROS action, upon which the following takes place:
   (a) Acquire an audio signal from the `/soundDetection/signal` ROS topic
   (b) Pass the audio signal through an automatic speech recognition (ASR) model to transcribe any speech utterances the audio signal contains to either Kinyarwanda or English text
   (c) Publish the transcribed text to the result ROS topic of the `/speechEvent/recognise_speech_action` ROS action
2. In case voice activity detection is unable to find speech utterances within a time period specified in the transcription request, an empty string is published to the result ROS topic of the `/speechEvent/recognise_speech_action` ROS action as soon as the specified time period elapses

The exact language between Kinyarwanda and English, which a running Speech Event ROS node transcribes, is decided based on a configuration option that is set during the initialisation stage when the ROS node is started. The language that is set during the initialisation of the Speech Event ROS node can be overridden by invoking the `/speechEvent/set_language` ROS service and passing a different language to it.

To supplement the core functionality that Speech Event performs (Kinyarwanda and English speech recognition), the text transcriptions that Speech Event transcribes are displayed on a graphical interface. The graphical interface is used as a replacement for the terminal interface, especially in presentation settings where displaying text transcriptions on the terminal is impractical due to the small font size used by the terminal and the extra verbosity of the results displayed on the terminal. This graphical interface is enabled when the ROS node is configured to run in verbose mode.

A ROS node that emulates Sound Detection is also availed, and its main purpose is to showcase the working of Speech Event in isolation, separated from the rest of the CSSR4Africa system and from the Pepper robot. This ROS node captures audio from the computer on which Speech Event is running on, and publishes the captured audio to the same ROS topic that Sound Detection publishes to (`/soundDetection/signal`), therefore mimicking Sound Detection's operation of acquiring audio from Pepper and publishing it to the aforementioned ROS topic.

It is also worth noting that Sound Detection may fail, and since the Behaviour Controller does not directly interface with Sound Detection, the Speech Event ROS node bears the burden of signaling failures that may arise in Sound Detection to the Behaviour Controller. This is done by publishing the message 'Error: soundDetection is down' on the result ROS topic of the `/speechEvent/recognise_speech_action` ROS action.

3 Module Specification

A running Speech Event ROS node performs speech-to-text as soon as a trigger is detected on the goal ROS topic of the `/speechEvent/recognise_speech_action` ROS action. An audio signal is acquired from the `/soundDetection/signal` ROS topic and any speech utterance contained in the sound signal is transcribed, and then the result published to the result topic of the `/speechEvent/recognise_speech_action` ROS action. The data that is input to Speech Event, the transformation that this data undergoes, and the data that Speech Event outputs is described below:

1. The language (either Kinyarwanda or English) that Speech Event is to be configured to operate in is set when the ROS node is started by setting a language configuration option
2. The language (either Kinyarwanda or English) set for Speech Event to operate in when it is first started can be overridden by making a `/speechEvent/set_language` ROS service call and passing a different language
3. When a trigger for the transcription process is detected on the goal ROS topic of the `/speechEvent/recogni` ROS action, the following takes place:
   (a) An audio signal published by the Sound Detection ROS node on the `/soundDetection/signal` ROS topic is acquired be Speech Event
   (b) The acquired audio signal is then passed through an ASR model that transcribes speech utterances present within the audio signal to a text string
   (c) The text string output by the ASR model is then published to the result ROS topic of the `/speechEvent/recognise_speech_action` ROS action

This process, and the position of Speech Event within the CSSR4Africa system architecture, is captured in Fig 2.

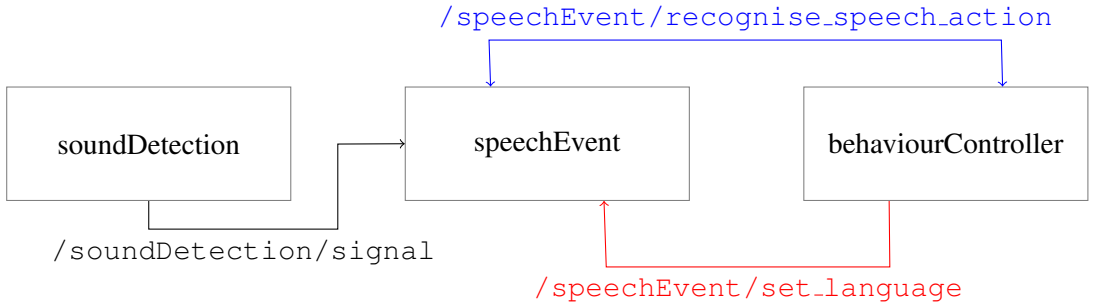

Figure 2: Speech Event within the CSSR4Africa architecture (`/soundDetection/signal` is a ROS topic, `/speechEvent/set_language` is a ROS service, and the bi-directional `/speechEvent/recognise_speech_action` is a ROS action)

4 Interface Design

4.1 Source Code

The file structure of Speech Event is as shown below:

```
speech_event/
├── action/
│   └── recognise_speech.action
├── config/
│   └── speech_event_configuration.ini
├── data/
│   └── pepper_topics.dat
├── models/
│   ├── silero_vad.jit
│   ├── stt_rw_conformer_transducer_large.nemo
│   ├── stt_en_conformer_transducer_large.nemo
│   ├── stt_rw_parakeet-tdt_ctc-110m.nemo
│   ├── stt_en_parakeet-tdt_ctc-110m.nemo
│   ├── stt_rw_whisper-large-v3-turbo.pt
│   └── stt_en_whisper-large-v3-turbo.pt
├── src/
│   ├── speech_event_application.py
│   ├── speech_event_implementation.py
│   ├── speech_event_gui.py
│   └── speech_event_utils.py
├── srv/
│   └── set_language.srv
├── README.md
└── speech_event_requirements.txt
```

The `action/` directory contains the `recognise_speech.action` action file specification for the ROS action `/speechEvent/recognise_speech_action` that is used to trigger a speech transcription process to run.

The `config/` directory houses configuration files that are used to configure different ROS nodes that are part of Speech Event. For Speech Event which contains just one ROS node, only the `speech_event_configuration.ini` file exists in this directory, and it is used to configure the main Speech Event ROS node once, at start up. The transcription language, which is configurable via this configuration file, can be updated at runtime by invoking the `/speechEvent/set_language` ROS service.

The `data/` directory contains data that is required by Speech Event when running. For Speech Event, only the `pepper_topics.dat` file is contained in this directory. This file contains a list of ROS topics that Speech Event listens to when it is running.

The `models/` directory holds seven deep learning models used to detect voice activity in audio signals and transcribe speech utterances:

1. Voice activity detection models
   (a) `silero_vad.jit`
2. Kinyarwanda speech recognition models
   (a) `stt_rw_conformer_transducer_large.nemo`
3. English speech recognition models
   (a) `stt_en_conformer_transducer_large.nemo`
   (b) `stt_en_parakeet-tdt_ctc-110m.nemo`
   (c) `stt_en_whisper-large-v3-turbo.pt`
4. Unused models (included as placeholders, but are just a copy of their English counterpart models)
   (a) `stt_rw_parakeet-tdt_ctc-110m.nemo`
   (b) `stt_rw_whisper-large-v3-turbo.pt`

When using a GPU-enabled computer, `stt_en_whisper-large-v3-turbo.pt` should be preferred for English speech recognition. It is the best of the three English models, but lags when run on CPU. For English speech recognition on CPU, `stt_en_parakeet-tdt_ctc-110m.nemo` should be preferred. It provides more accurate transcriptions when compared to `stt_en_conformer_transducer_large.nemo`, which can also be used on CPU. For Kinyarwanda speech recognition on both GPU and CPU, `stt_rw_conformer_transducer_large.nemo` should be used.

The `src/` directory holds all the Python source code files that implement the Speech Event system.

The `srv/` directory contains the `set_language.srv` service file specification for the ROS service `/speechEvent/set_language`.

The file `README.md` contains documentation of how to use Speech Event, and the Python requirements file `speech_event_requirements.txt` contains a list of versioned Python packages that Speech Event depends on.

4.2 Configuration Files

A Speech Event ROS node needs to be configured prior to starting it, a task that is accomplished via a configuration file. The configuration parameters are displayed in Table 1.

Key	Value	Description
`language`	`kinyarwanda`, `english`	Specifies the language in which the utterance is spoken.
`model`	`conformer-transducer`, `parakeet`, `whisper`	Specifies the ASR model to be used.
`verboseMode`	`true`, `false`	Specifies whether diagnostic data is to be printed to the terminal.
`cuda`	`true`, `false`	Specifies whether to use GPUs. The term 'cuda' is chosen as the key to alert the user that only NVIDIA GPUs are supported.
`confidence`	`<float>`	The confidence level on a scale of 0 to 1 above which transcriptions are assumed to be acceptable and correct.
`vadThreshold`	`<float>`	The confidence level on a scale of 0 to 1 above which an utterance is considered valid in an audio signal.
`sampleRate`	`<integer>`	Specifies the sampling rate of the incoming audio sourced from the /soundDetection/signal ROS topic.
`interUtteranceLen`	`<float>`	The minimum length (in seconds) between two successsive utterances, below which the two are assumed to be one utterance.
`minUtteranceLen`	`<float>`	The minimum length (in seconds) of an utterance; shorter utterances are ignored and therefore discarded.
`maxUtteranceLength`	`<integer>`	The maximum length (in seconds) of an utterance, after which any extra utterances are truncated.
`heartbeatMsgPeriod`	`<integer>`	Specifies the time period in seconds at which a periodic heartbeat message is sent to the terminal.

Table 1: Speech Event configuration file options

4.3 Data Files

The topics that Speech Event subscribes to are listed in the `pepper_topics.dat` file, as shown in Table 2.

Key	Value
`soundDetection`	`/soundDetection/signal`

Table 2: ROS topics that Speech Event subscribes to

4.4 Topics Subscribing To

A running Speech Event ROS node acquires audio signals from the `/soundDetection/signal` ROS topic. Table 3 shows this fact, while also mentioning the ROS node that publishes these audio signals.

Topic	Node	Platform
`/soundDetection/signal`	`soundDetection`	`Physical robot`

Table 3: Topics subscribing to

4.5 Topics Publishing To

The Speech Event ROS node does not publish to any independent ROS topics, but contains one ROS action that advertises its cancel, feedback, goal, result, and status ROS topics.

4.6 Services Supported

A running Speech Event ROS node can have its language of operation (Kinyarwanda or English) updated by invoking the `/speechEvent/set_language` ROS service. Table 4 elaborates more about this ROS service.

Service	Message Value	Effect
`/speechEvent/set_language`	`kinyarwanda`, `english`	Set the language to either Kinyarwanda or English

Table 4: Services supported

4.7 Actions Supported

A running Speech Event ROS node depends on triggers on the goal ROS topic of the `/speechEvent/recognise_speech_action` ROS action to perform speech recognition. Table 5 elaborates more about this ROS action. This ROS action advertises the following ROS topics:

1. `/speechEvent/recognise_speech_action/cancel`
2. `/speechEvent/recognise_speech_action/feedback`
3. `/speechEvent/recognise_speech_action/goal`
4. `/speechEvent/recognise_speech_action/result`
5. `/speechEvent/recognise_speech_action/status`

Action	Function
`/speechEvent/recognise_speech_action`	Handles the speech recognition process

Table 5: Services supported

5 Module Design

5.1 Voice Activity Detection

Voice Activity Detection (VAD) is the process of investigating whether speech is present within an audio segment. In Speech Event, this process is conducted to ascertain that only audio segments with human speech are handed over for speech recognition. The speech recognition process is compute intensive, and if all audio segments (including those with no human speech) are handed over for speech recognition, then a waste in compute resource and a lagging effect are observed.

The Silero voice activity detector [2] is used in Speech Event. It is a deep learning based detector, primarily employing convolutional layers interspaced with Long Short-Term Memory (LSTM) layers. The final layer is a classification head that outputs a probability in the range [0.0, 1.0] to quantify the likelihood of whether human speech is present within an audio segment, 1.0 being full certainty and 0.0 being complete uncertainty that human speech is present. Based on experimentation within a specific environment, a threshold value such as 0.5 is selected such that whenever the predicted probability is greater than this threshold, then speech utterances are assumed to be contained within the audio segment passed to the model. Not much about the model architecture is public (based on this GitHub discussion: https://github.com/snakers4/silero-vad/discussions/107), and therefore we will not be discussing it in further detail.

5.2 Automatic Speech Recognition

Both the Kinyarwanda and English speech recognition processes performed by Speech Event rely on deep learning models. Three deep learning models are supported by Speech Event: NVIDIA's NeMo conformer transducer large, NVIDIA's NeMo Parakeet TDT-CTC (Token-and-Duration Transducer and Connectionist Temporal Classification), and OpenAI's Whisper Large v3 Turbo.

5.2.1 Conformer Transducer Large

The conformer transducer architecture is made up of a conformer encoder and a transducer decoder. The conformer encoder combines transformers and Convolution Neural Networks (CNNs) in an attempt to utilise the strengths of both, with transformers excelling at capturing global dependencies of an audio signal while CNNs excel at capturing local dependencies of an audio signal [3]. The transducer decoder, on the other hand, makes use of Recurrent Neural Networks (RNNs) to form an auto-regressive model whose next token prediction is conditioned on the previously predicted tokens.

5.2.1.1 Preprocessor

When an audio signal is received, it is first passed through a preprocessor block that extracts filterbank features from the audio signal. For this process, an audio signal is first converted to a spectrogram by a Short Time Fourier Transform (STFT), and then a series of filters is applied to the resultant spectrogram to obtain the filterbank features that each represent the magnitude of the energy within a distinct frequency band. This preprocessor block is visualised in Fig 3.

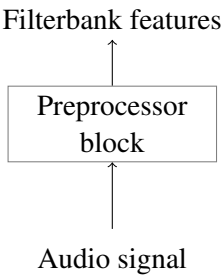

Figure 3: Audio to filterbank features preprocessor

5.2.1.2 Encoder Architecture

The filterbank features obtained from the preprocessor block are then passed to the encoder part of the model. The encoder transforms acoustic features in the original audio signal captured within the filterbank features into latent features that better represent the speech in the original audio signal.

A conformer encoder is shown in Fig 4. The encoder used in Speech Event has 17 conformer blocks.

The filterbank features are first passed through a SpecAugment block. SpecAugment is a data augmentation method that is suitable for end-to-end speech recognition models such as the Conformer Transducer model used by Speech Event. “The augmentation policy consists of warping the features, masking blocks of frequency channels, and masking blocks of time steps” [4].

The features are then processed by a convolution subsampling block, dropout applied, and then passed through a series of 17 conformer blocks in the encoder.

Each confomer block consists of a multi-head self-attention module and a convolution module collectively sandwiched between two feedforward modules [3]. The self-attention module is more adept at capturing global feature dependencies, while the convolution module is more adept at capturing local feature dependencies, thereby capturing the semantics of speech utterances much better than models that only specialise in capturing only one of either the global or local feature dependencies.

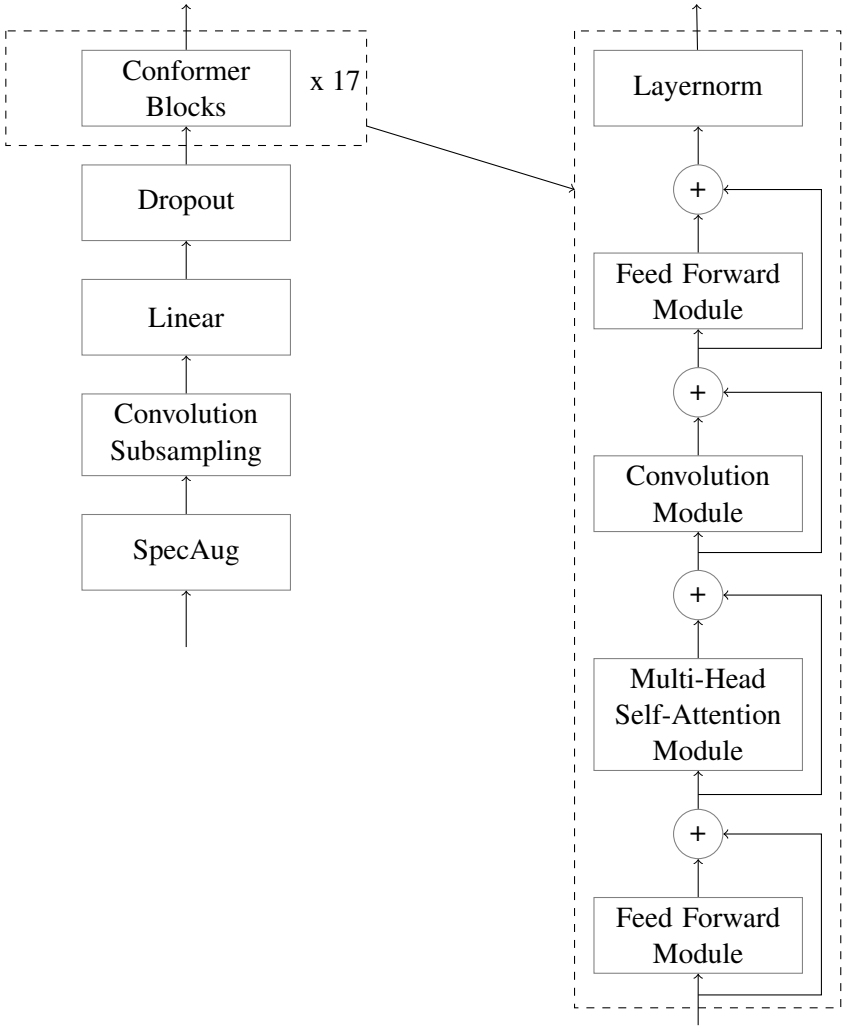

Figure 4: Conformer encoder architecture [3]

5.2.1.3 Decoder/Prediction Network Architecture

Keeping in line with Gulati et al., a single LSTM layer is used in the decoder [3]. An embedding layer converts input tokens into vectorised representations that are then passed to the LSTM layer. Additionally, drop out is applied both in the LSTM layer and on the output of the LSTM layer.

The decoder architecture is shown in Fig 5.

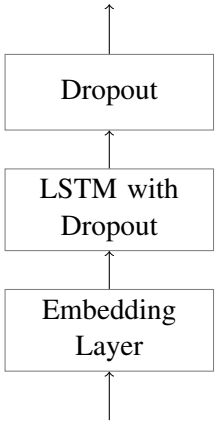

Figure 5: Transducer predictor architecture

5.2.1.4 Joint Network Architecture

The joint network combines the output of the encoder and the output of the decoder in order to predict the text in the speech utterances contained in the original sound signal passed to the encoder. The combined outputs are then passed through a ReLU activation, dropout is applied, and lastly passed through a linear layer.

The joint network architecture is shown in Fig 6.

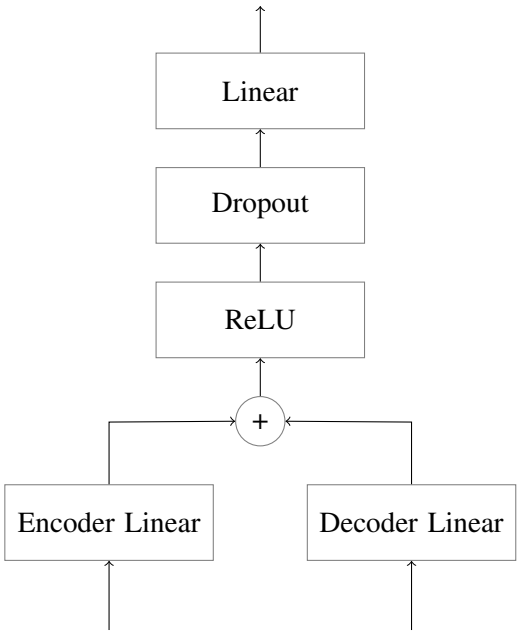

Figure 6: Joint architecture

5.2.1.5 Combined Model Architecture

The four different sections of the full conformer transducer architecture discussed above are combined together to form the whole network that performs end-to-end automatic speech recognition. This full model's architecture is summaried in Fig 7.

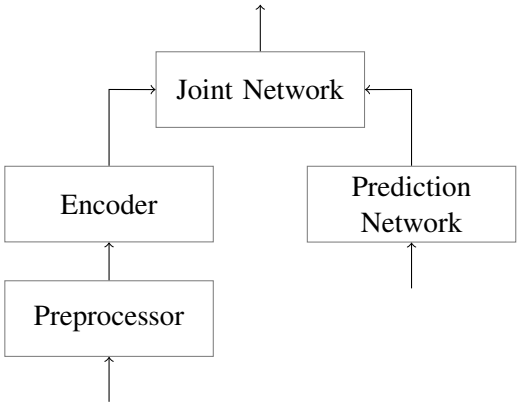

Figure 7: Combined model architecture

5.2.2 Parakeet TDT-CTC 110M

Parakeet TDT-CTC (Token-and-Duration Transducer and Connectionist Temporal Classification) is a family of models that improves on the conformer trasducer architecture, replacing the conformer encoder with a fast confomer encoder [5] for faster inference (2.8 times faster than conformer), and employing the use of TDT [6] which facilitates joint prediction of both tokens and their durations that enables the model to have faster inference times (upto 2.82 times faster) and higher accuracy than transducer-based models.

5.2.2.1 Preprocessor

When an audio signal is received, it is first passed through a preprocessor block that extracts Mel spectrogram features from the audio signal. This preprocessor block is similar to the conformer transducer's preprocessor block that is visualised in Fig 3.

5.2.2.2 Encoder Architecture

The Mel spectrogram features obtained from the preprocessor block are then passed to the encoder part of the model. The encoder transforms acoustic features in the original audio signal captured within the Mel spectrogram features into latent features that better represent the speech in the original audio signal. However, unlike the conformer blocks used in the conformer transducer, fast conformer blocks are used instead. Fast conformer blocks have the following improvements over conformer blocks, copied verbatim from [5]:

1. 8x downsampling at the start of the encoder, thereby reducing the compute cost of subsequent attention layers by 4x

2. Replacement of the original convolution sub-sampling layers with depthwise separable convolutions
3. Reduction of the number of convolutional filters in the downsampling block to 256
4. Reduction of the convolutional kernel size to 9

Extra modifications have also been applied to enable fast conformer encoders process very long speech utterances. These modifications see the fast conformer combine local attention with a global context token to process very long speech utterances.

5.2.2.3 Decoder/Prediction Network Architecture

The decoder architecture used by Parakeet TDT-CTC is similar to that used by the conformer transducer model, as shown in Fig 5.

5.2.2.4 CTC Decoder

This is an extra decoder that projects the encoder's output directly to the vocabulary size and therefore providing additional supervision during training. Speech Event does not use this component during inference.

5.2.2.5 Joint Network Architecture

As in the conformer transducer model, the joint network combines the output of the encoder and the output of the decoder in order to predict the text in the speech utterances contained in the original sound signal passed to the encoder. However, unlike the joint network used in the conformer transducer model, parakeet TDT-CTC emits two outputs: one for the output token, and the other for the duration of the token. [6]

The joint network workings are shown in Fig 8.

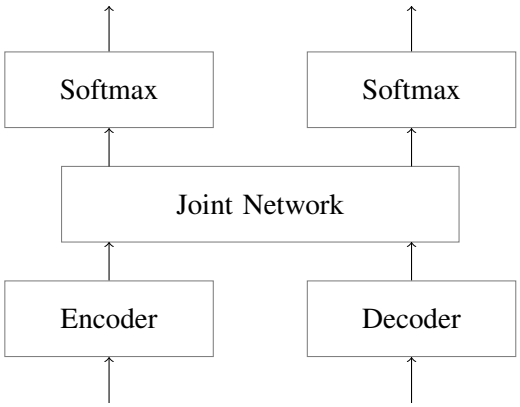

Figure 8: Joint network's interactions with the encoder and decoder, and the two outputs it emits - a token and its duration

5.2.2.6 Combined Model Architecture

The four different sections of the full Parakeet TDT-CTC architecture discussed above are combined together to form the whole network that performs end-to-end automatic speech recognition. This full model's architecture is summaried in Fig 9.

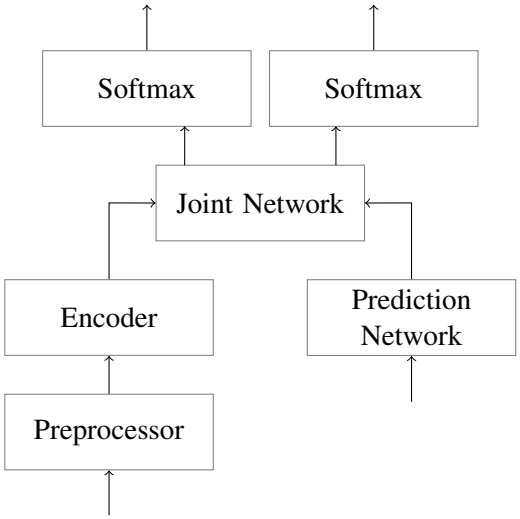

Figure 9: Combined model architecture

5.2.3 Whisper Large v3 Turbo

Whisper Large v3 Turbo is a variant of OpenAI's Whisper family of models [7], specifically a fine-tuned Whisper Large v3 model with a pruned decoder for faster auto-regressive generation. Whisper models are encoder-decoder transformer models. The encoder takes as input log-Mel spectrograms and generates a vector representation that is then passed via cross-attention to the decoder, which auto-regressively generates a text transcript. Whisper's model architecture is shown in Fig 10.

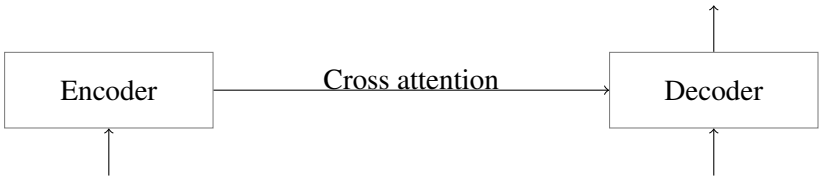

Figure 10: Whisper model architecture

6 Testing

The `unit_tests` package contains unit tests for all CSSR4Africa ROS nodes, including Speech Event. Python's unittest testing framework is used for testing Speech Event. The unittest package is part of the standard Python library, and therefore no extra packages need to be installed when running tests.

These tests check to ensure that Speech Event's transcription process works as expected end-to-end, focusing on ensuring that the `/speechEvent/recognise_speech_action` ROS action is fully functional. They also check that the `/speechEvent/set_language` ROS service work as expected.

A test report is generated in the `data/` directory, `speech_event_test_output.dat`. It shows all the tests that have been run and whether they have passed or failed. For tests that err, the reason for the error is also shown within the test report.

Before running tests, the ROS node that runs these tests has to be configured. The driver ROS node that emulates the Sound Detection ROS node also has to be configured, if the tests will not be run using the Pepper robot.

The configuration options for the ROS node that runs the tests are located in the configuration file `speech_event_test_configuration.ini`. They are shown in Table 6.

Key	Value	Description
`heartbeatMsgPeriod`	`<integer>`	Specifies the time period in seconds at which a periodic heartbeat message is sent to the terminal.
`waitTimeout`	`<integer>`	Specifies how long to wait for speechEvent to start.
`mode`	`mic`, `file`	Use 'mic' when using real-time utterances from Pepper or the driver ROS node, and 'file' when using saved audio files.

Table 6: Speech Event Test configuration file

When using a driver ROS node, the following is important when configuring the driver ROS node:

- Tweak the `speechAmplitudeThreshold` in case the driver is having trouble capturing speech utterances. Increase it in case speech is not being detected, and decrease it in case ambient noise is being captured together with the speech utterances. You may also tweak the sensitivity of the PC's microphones in case `sspeechAmplitudeThreshold` fails to solve the aforementioned issues that may arise.
- When using the driver to test Speech Event, ensure that `mode` in the driver configuration file matches `mode` in Speech Event Test configuration file (either set both to `file` or set both to `mic`). When using a running Sound Detection ROS node to test Speech Event, ensure that `mode` in the Speech Event Test configuration is set to `mic`.
- If using ROS launch files to launch the test harness, the argument passed to the `mode` parameter of the launch file will override whatever is set in the configuration files. Therefore when using

the test harness launch file, no need to update the `mode` option in the configuration files, as setting it once while launching using the test harness launch file suffices.

The configuration options for the driver ROS node are located in the driver configuration file `speech_event_driver_configuration.ini`. They are shown in Table 7.

Key	Value	Description
`chunkSize`	`<integer>`	Number of samples to acquire at a go every time that audio is read from the PC's microphones.
`sampleRate`	`<integer>`	The sample rate to use when acquiring audio from the PC's microphones.
`speechAmplitudeThreshold`	`<float>`	Threshold above which a signal sample is assumed to contain a speech utterance.
`mode`	`mic`, `file`	Use 'mic' when using real-time utterances from Pepper or the driver ROS node, and 'file' when using saved audio files.

Table 7: Speech Event Driver configuration file

The tests for Speech Event test the following:

1. The working of the `/speechEvent/set_language` ROS service
2. The end-to-end working of Speech Event's transcription process via the `/speechEvent/recognise_speech_action` ROS action

A sample test report that is generated at the end of each testing process is displayed below (some lines have been artificially folded to the next line in this report to avoid overflowing out of the page):

```
Speech Event Test Report

Date: 2026-04-24 16:18:08


============================================================

Test SpeechEvent's /speechEvent/set_language ROS service
---
As a:     behaviourController developer
I want:   a means of setting the transcription language of
            speechEvent at runtime
So that:  I don't have to restart speechEvent every time I
            update the transcription language
---
Set unsupported language (Kiswahili): PASS
```

```
Set unsupported language (Arabic): PASS
Set supported language (English): PASS
Set supported language (Kinyarwanda): PASS

============================================================

Test SpeechEvent's end-to-end transcription process
---
Given:    a running speechEvent ROS node
When:     a goal is sent to the /speechEvent/recognise_speech_action
            ROS action
Then:     transcribe utterances in audio published on the
            /soundDetection/signal ROS topic and return
            the result via the /speechEvent/recognise_speech_action
            ROS action
---

Transcribe 'rw-ingendo': PASS
Transcribe 'rw-atandatu': PASS
Transcribe 'rw-ibikenewe': PASS
Transcribe 'rw-kabiri': PASS
Transcribe 'en-turner construction was the construction manager
    for the project': PASS
Transcribe 'en-he now resides in monte carlo': PASS
Transcribe 'en-the council was held': PASS
```

7 User Manual

7.1 Installation

The Speech Event ROS node needs to be installed before it can be used. To install Speech Event, clone the CSSR4Africa repository where it is housed into the CSSR4Africa ROS workspace, and then proceed with the following steps (the `README.md` file located within the Speech Event ROS node's directory of the cloned repository contains a similar but more elaborative list of installation steps):

1. Download ASR model files and move them to the correct directory (skip if you already have the ASR models in the correct directory):

```
# Install git-lfs
sudo apt-get update
sudo apt-get install git-lfs
git lfs install

# Clone CSSR4Africa models repository from Hugging Face
git clone https://huggingface.co/cssr4africa/cssr4africa_models

# Move speech event models to the models/ directory
mv cssr4africa_models/speech_event/models/* $HOME/workspace/
    ↪ pepper_rob_ws/src/cssr4africa/cssr_system/speech_event/models
```

2. Install required Linux tools:

```
# Install required Linux packages
sudo apt-get update
sudo apt-get install cython3 ffmpeg gfortran libopenblas-dev
    ↪ libopenblas64-dev patchelf pkg-config portaudio19-dev python3-
    ↪ testresources python3-tk python3-typing-extensions sox
```

3. Create a Python virtual environment and install required Python packages (Speech Event has been tested and proven to work using Python3.8):

```
# Create CSSR4Africa's virtual environments' parent directory if it
    ↪ doesn't exist
mkdir -p $HOME/workspace/pepper_rob_ws/src/cssr4africa_virtual_envs

# Change the current working directory to CSSR4Africa's virtual
    ↪ environments' parent directory
cd $HOME/workspace/pepper_rob_ws/src/cssr4africa_virtual_envs

# Create speech event's Python virtual environment
python3 -m venv cssr4africa_speech_event_env

# Source the created virtual environment
source cssr4africa_speech_event_env/bin/activate

# Install required Python dependencies
pip install -r $HOME/workspace/pepper_rob_ws/src/cssr4africa/
    ↪ cssr_system/speech_event/speech_event_requirements.txt
```

4. Build the ROS workspace (it's best to proceed with the next steps on a separate terminal, otherwise an error may arise when running 'catkin_make'):

```
# Change the current working directory to CSSR4Africa's ROS
    ↪ workspace
cd $HOME/workspace/pepper_rob_ws

# Build the CSSR4Africa project
catkin_make

# Source the CSSR4Africa ROS workspace
source devel/setup.bash
```

5. Make application files executable:

```
# Add executable permission flag to speech event's application file
chmod +x $HOME/workspace/pepper_rob_ws/src/cssr4africa/cssr_system/
    ↪ speech_event/src/speech_event_application.py

# Add executable permission flag to speech event's test application
    ↪ file
chmod +x $HOME/workspace/pepper_rob_ws/src/cssr4africa/unit_tests/
    ↪ speech_event_test/src/speech_event_test_application.py

# Add executable permission flag to speech event's driver file
chmod +x $HOME/workspace/pepper_rob_ws/src/cssr4africa/unit_tests/
    ↪ speech_event_test/src/speech_event_driver.py
```

7.2 Usage

To run a Speech Event ROS node, a Sound Detection ROS node needs to also be running (or a Speech Event driver node, which is described in a subsequent subsection).

1. Run a Speech Event ROS node: `rosrun speech_event speech_event_application.py`
2. View text transcriptions on the terminal (optional): `rostopic echo /speechEvent/recognise_speech_action/result`

If running the Speech Event ROS node in verbose mode (by setting the configuration option `verboseMode` to `true`), a graphical application that displays the text transcriptions will open on the screen.

7.3 Driver ROS Node

A driver ROS node that mimics the Sound Detection ROS node is also provided. It capturing audio from a personal computer's microphone instead of from Pepper, and publishes the audio signal on the `/soundDetection/signal` ROS topic in the same way that the Sound Detection ROS node does. Execute the command `rosrun speech_event speech_event_driver.py` to bring up this driver ROS node.

References

[1] Cristina Romero-González, Jesus Martínez-Gómez, and Ismael García-Varea. Spoken language understanding for social robotics. In *2020 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pages 152–157, April 2020.

[2] Silero Team. Silero vad: pre-trained enterprise-grade voice activity detector (vad), number detector and language classifier. `https://github.com/snakers4/silero-vad`, 2024.

[3] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented Transformer for Speech Recognition. In *Interspeech 2020*, pages 5036–5040. ISCA, October 2020.

[4] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. In *Interspeech 2019*, pages 2613–2617. ISCA, September 2019.

[5] Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. Fast conformer with linearly scalable attention for efficient speech recognition, 2023.

[6] Hainan Xu, Fei Jia, Somshubra Majumdar, He Huang, Shinji Watanabe, and Boris Ginsburg. Efficient sequence transduction by jointly predicting tokens and durations, 2023.

[7] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine Mcleavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28492–28518. PMLR, 23–29 Jul 2023.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

Clifford Onyonka, Carnegie Mellon University Africa.
David Vernon, Carnegie Mellon University Africa.
Richard Muhirwa, Carnegie Mellon University Africa.

Document History

Version 1.0

First draft.
Clifford Onyonka.
20 February 2025.

Version 1.1

Added a ROS service that will be used by the Behaviour Controller to set the language (Kinyarwanda or English) that Speech Event will operate in.
Clifford Onyonka.
13 March 2025.

Version 1.2

Updated configuration file keys to camel case from snake case
Added the 'confidence', 'speechPausePeriod', and 'maxUtteranceLength' keys to the Speech Event ROS node's configuration file
Added a ROS service that will be used by the Behaviour Controller to enable or disable Speech Event's transcription process
Updated the testing section to use use a test ROS node to run tests
Updated the user manual section to detail usage of Speech Event using a Python virtual environment and steps for downloading ASR models from Hugging Face
Clifford Onyonka.
11 June 2025.

Version 1.3

Fixed typing errors.
Use listings for code blocks.
Clifford Onyonka.
30 July 2025.

Version 1.4

Use a ROS action for speech transcription
Update Speech Event ROS node and Speech Event driver ROS node configuration file options
Add Voice Activity Detection (VAD)
Add Parakeet TDT-CTC and Whisper Large v3 Turbo speech recognition models
Clifford Onyonka.
07 May 2026.